

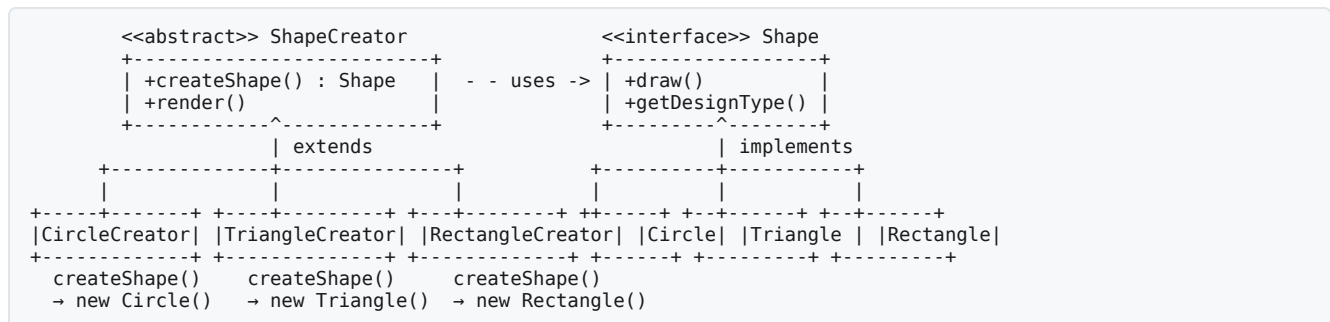
Factory Method Design Pattern

Intent

Define an interface for creating an object, but let **subclasses decide which concrete class to instantiate**. The creator's algorithm (`render()`) works against the abstract product (`Shape`); the *factory method* (`createShape()`) is the single point each subclass overrides to supply its own product.

This is the **canonical GoF Factory Method**: an abstract `ShapeCreator` with one abstract creation step, and one concrete creator per product. The client picks a creator, never a product — it calls `new` on `CircleCreator`, but never on `Circle`.

UML class diagram



The players

```
enums/DesignType
factory/contract/Shape
factory/contract/concret/Circle
                        /Rectangle
                        /Triangle
factory/ShapeCreator
factory/creators/CircleCreator
                /RectangleCreator
                /TriangleCreator
```

each product's self-declared identity (CIRCLE, TRIANGLE, RECTANGLE)
the product interface
concrete products

the abstract creator – declares `createShape()`, owns `render()`
concrete creators – one per product, each overrides `createShape()`

The code, line by line

Shape — the product interface

```
public interface Shape {
    DesignType getDesignType();
    void draw();
}
```

- `draw()` is the behavior every product offers; it's what the creator's `render()` ultimately calls.
- `getDesignType()` lets each product declare its own identity (`Circle` says `CIRCLE`), so nothing outside the product needs a mapping table.

Circle / Rectangle / Triangle — concrete products

```
public class Circle implements Shape {
    @Override public void draw() { System.out.println("Shape is circle."); }
    @Override public DesignType getDesignType() { return DesignType.CIRCLE; }
}
```

- Plain classes — the pattern needs nothing more. Only the concrete *creator* knows which one gets instantiated.

ShapeCreator — the abstract creator

```
public abstract class ShapeCreator {  
    public abstract Shape createShape();  
  
    public void render() {  
        Shape shape = createShape();  
        shape.draw();  
    }  
}
```

- `createShape()` **is the factory method** — the one deferred decision. It returns the abstract `Shape`, so nothing in this class ever names a concrete product.
- `render()` is the **template step** that makes the pattern useful: it contains the creator's real algorithm (create, then draw). Subclasses inherit the algorithm and customize only the creation step. If `render()` grew (validate → create → draw → log), every creator would gain those steps for free.

CircleCreator / TriangleCreator / RectangleCreator — concrete creators

```
public class CircleCreator extends ShapeCreator {  
    @Override  
    public Shape createShape() {  
        return new Circle();  
    }  
}
```

- One line of real logic: `new Circle()`. This is the **only place in the module where a concrete product is constructed**. The other two creators are identical except for the product they `new`.
- The return type stays `Shape` (not `Circle`) — callers get the abstraction even from the concrete creator.

Why the design decisions

Why an abstract class + subclasses instead of one factory with a `switch/map`?

Because the *variation point is the creation itself*, and Factory Method places that variation in the type system:

- Adding `Hexagon` = add `Hexagon` implements `Shape` + `HexagonCreator` extends `ShapeCreator`. **No existing file is edited** — Open/Closed by construction, no central registry to grow.
- Each creator can later specialize more than construction (cache its product, pre-configure it, read settings) without touching its siblings.
- A key-indexed factory (enum → instance map) answers a different question — "give me the product for this key at runtime." That variant belongs to a *parameterized* factory; this module demonstrates the GoF subclassing form.

Why does `render()` live in the abstract creator?

That's the point of the pattern: **the creator is not just a maker, it's a user of its own product**. `render()` is code written once against `Shape` that works for every current and future subclass. The factory method exists so this shared algorithm can create the right product without knowing its class.

Why does `createShape()` return `Shape` and not the concrete type?

So the inherited `render()` — and any caller — stays coupled only to the abstraction. The concrete class name appears exactly once, inside its creator.

Why do products still carry `getDesignType()` ?

Each product self-describes its identity. Nothing in the creator flow needs it, but it keeps a product's key with the product itself — the single source of truth if a lookup table (or serialization) is ever layered on top.

Execution flow (as run from `main`)

```
DesignPatternsApplication.main  
├── SpringApplication.run(...)      boots the app (the pattern itself doesn't need Spring)  
└── List.of(new CircleCreator(), new TriangleCreator(), new RectangleCreator())
```

```
└─ the client chooses CREATORS, never products
└─ creators.forEach(ShapeCreator::render)
    └─ CircleCreator.render()
        └─ createShape() → new Circle() → draw() → "Shape is circle."
    └─ TriangleCreator.render()
        └─ createShape() → new Triangle() → draw() → "Shape is triangle."
    └─ RectangleCreator.render()
        └─ createShape() → new Rectangle() → draw() → "Shape is rectangle."
```

Factory Method vs. Abstract Factory (so you don't confuse them)

- **Factory Method (this module)** — one product per creator, and the *subclass* is the decision. You pick the factory (`CircleCreator`), and it makes its one product.
- **Abstract Factory** (the sibling module) — a **matched set** of related products (a chair *and* a table that belong together) behind one factory interface.

Rule of thumb: if choosing the factory chooses one product, it's Factory Method; if choosing the factory chooses a whole family, it's Abstract Factory.